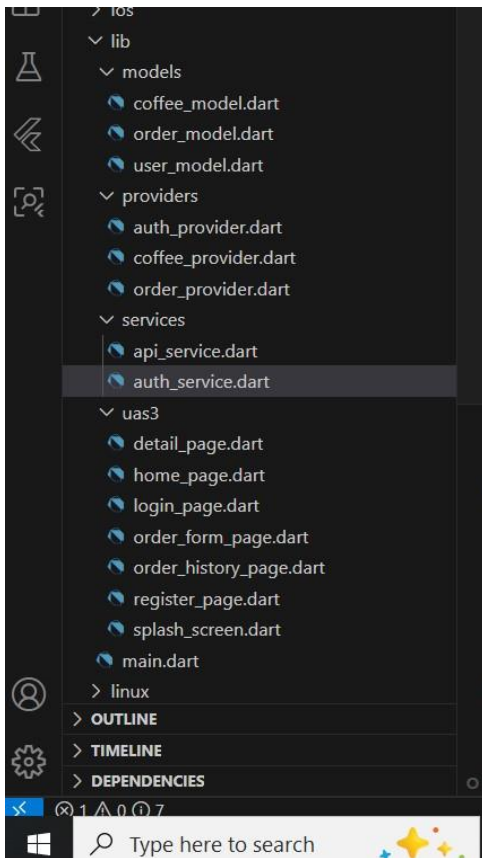


Ini folder coffe_day yg sudah dibuat file flutter dan php native nya sama dixampp pun database sudah dibuat

Ini code flutter nya



1.models

a.coffe_model.dart

```
class Coffee {
  final int id;
  final String name;
  final String description;
  final int price;
  final String image;

  Coffee({
    required this.id,
    required this.name,
    required this.description,
    required this.price,
    required this.image,
  });

  factory Coffee.fromJson(Map<String, dynamic> json) {
    return Coffee(
      id: json['id'],
      name: json['name'],
```

```

        description: json['description'],
        price: json['price'],
        image: json['image'],
    );
}

Map<String, dynamic> toJson() {
    return {
        'id': id,
        'name': name,
        'description': description,
        'price': price,
        'image': image,
    };
}
}

```

b.order_model.dart

```

class Order {
    final int id;
    final int userId;
    final String itemName;
    final String imagePath;
    final int total;
    final DateTime date;
    final String paymentMethod;

    Order({
        required this.id,
        required this.userId,
        required this.itemName,
        required this.imagePath,
        required this.total,
        required this.date,
        required this.paymentMethod,
    });

    factory Order.fromJson(Map<String, dynamic> json) {
        return Order(
            id: json['id'],
            userId: json['user_id'],
            itemName: json['item_name'],
            imagePath: json['image_path'],
            total: json['total'],
            date: DateTime.parse(json['date']),
            paymentMethod: json['payment_method'],
        );
    }
}

```

c.user_model.dart

```
class User {
  final int? id;
  final String name;
  final String email;
  final String? password;

  User({
    this.id,
    required this.name,
    required this.email,
    this.password,
  });

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['user_id'],
      name: json['name'],
      email: json['email'],
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'name': name,
      'email': email,
      'password': password,
    };
  }
}
```

2.providers

a.auth_provider..dart

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import '../models/user_model.dart';
import '../services/auth_service.dart';

class AuthProvider with ChangeNotifier {
  User? _user;
  bool _isLoading = false;
  String _errorMessage = '';

  User? get user => _user;
  bool get isLoading => _isLoading;
  String get errorMessage => _errorMessage;

  final AuthService _authService = AuthService();

  Future<bool> login(String email, String password) async {
```

```

        _isLoading = true;
        notifyListeners();

        try {
            final user = await _authService.login(email, password);
            _user = user;
            await _saveUserData(user);
            _errorMessage = '';
            _isLoading = false;
            notifyListeners();
            return true;
        } catch (e) {
            _errorMessage = e.toString();
            _isLoading = false;
            notifyListeners();
            return false;
        }
    }

Future<bool> register(String name, String email, String password) async {
    _isLoading = true;
    notifyListeners();

    try {
        final user = await _authService.register(name, email, password);
        _user = user;
        await _saveUserData(user);
        _errorMessage = '';
        _isLoading = false;
        notifyListeners();
        return true;
    } catch (e) {
        _errorMessage = e.toString();
        _isLoading = false;
        notifyListeners();
        return false;
    }
}

Future<void> logout() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.clear();
    _user = null;
    notifyListeners();
}

Future<void> loadUser() async {
    final prefs = await SharedPreferences.getInstance();
    final userId = prefs.getInt('user_id');
    final userName = prefs.getString('name');
    final userEmail = prefs.getString('email');

```

```

    if (userId != null && userName != null && userEmail != null) {
      _user = User(id: userId, name: userName, email: userEmail);
      notifyListeners();
    }
  }
}

Future<void> _saveUserData(User user) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setInt('user_id', user.id!);
  await prefs.setString('name', user.name);
  await prefs.setString('email', user.email);
}
}

```

b.coffe_provider.dart

```

import 'package:flutter/material.dart';
import '../models/coffee_model.dart';
import '../services/api_service.dart';

class CoffeeProvider with ChangeNotifier {
  List<Coffee> _coffees = [];
  bool _isLoading = false;
  String _errorMessage = '';

  List<Coffee> get coffees => _coffees;
  bool get isLoading => _isLoading;
  String get errorMessage => _errorMessage;

  final ApiService _apiService = ApiService();

  Future<void> fetchCoffees() async {
    _isLoading = true;
    notifyListeners();

    try {
      _coffees = await _apiService.getCoffees();
      _errorMessage = '';
    } catch (e) {
      _errorMessage = e.toString();
    }

    _isLoading = false;
    notifyListeners();
  }

  Future<void> addCoffee(Coffee coffee) async {
    _isLoading = true;
    notifyListeners();
  }
}

```

```

    try {
      final newCoffee = await _apiService.addCoffee(coffee);
      _coffees.add(newCoffee);
      _errorMessage = '';
    } catch (e) {
      _errorMessage = e.toString();
    }

    _isLoading = false;
    notifyListeners();
  }

Future<void> updateCoffee(Coffee coffee) async {
  _isLoading = true;
  notifyListeners();

  try {
    final updatedCoffee = await _apiService.updateCoffee(coffee);
    final index = _coffees.indexWhere((c) => c.id == coffee.id);
    if (index != -1) {
      _coffees[index] = updatedCoffee;
    }
    _errorMessage = '';
  } catch (e) {
    _errorMessage = e.toString();
  }

  _isLoading = false;
  notifyListeners();
}

Future<void> deleteCoffee(int id) async {
  _isLoading = true;
  notifyListeners();

  try {
    await _apiService.deleteCoffee(id);
    _coffees.removeWhere((coffee) => coffee.id == id);
    _errorMessage = '';
  } catch (e) {
    _errorMessage = e.toString();
  }

  _isLoading = false;
  notifyListeners();
}
}

```

c.order_provider.dart

```

import 'package:flutter/material.dart';
import '../models/order_model.dart';
import '../services/api_service.dart';

class OrderProvider with ChangeNotifier {
  List<Order> _orders = [];
  bool _isLoading = false;
  String _errorMessage = '';

  List<Order> get orders => _orders;
  bool get isLoading => _isLoading;
  String get errorMessage => _errorMessage;

  final ApiService _apiService = ApiService();

  Future<void> fetchOrders(int userId) async {
    _isLoading = true;
    notifyListeners();

    try {
      _orders = await _apiService.getOrders(userId);
      _errorMessage = '';
    } catch (e) {
      _errorMessage = e.toString();
    }

    _isLoading = false;
    notifyListeners();
  }

  Future<void> addOrder(Order order) async {
    _isLoading = true;
    notifyListeners();

    try {
      final newOrder = await _apiService.addOrder(order);
      _orders.add(newOrder);
      _errorMessage = '';
    } catch (e) {
      _errorMessage = e.toString();
    }

    _isLoading = false;
    notifyListeners();
  }
}

```

3.services

a.api_services.dart

```

import 'package:flutter/material.dart';

```

```

import '../models/order_model.dart';
import '../services/api_service.dart';

class OrderProvider with ChangeNotifier {
  List<Order> _orders = [];
  bool _isLoading = false;
  String _errorMessage = '';

  List<Order> get orders => _orders;
  bool get isLoading => _isLoading;
  String get errorMessage => _errorMessage;

  final ApiService _apiService = ApiService();

  Future<void> fetchOrders(int userId) async {
    _isLoading = true;
    notifyListeners();

    try {
      _orders = await _apiService.getOrders(userId);
      _errorMessage = '';
    } catch (e) {
      _errorMessage = e.toString();
    }

    _isLoading = false;
    notifyListeners();
  }

  Future<void> addOrder(Order order) async {
    _isLoading = true;
    notifyListeners();

    try {
      final newOrder = await _apiService.addOrder(order);
      _orders.add(newOrder);
      _errorMessage = '';
    } catch (e) {
      _errorMessage = e.toString();
    }

    _isLoading = false;
    notifyListeners();
  }
}

```

b.auth_service.dart

```

import 'package:flutter/material.dart';
import '../models/order_model.dart';
import '../services/api_service.dart';

```



```

class OrderProvider with ChangeNotifier {
  List<Order> _orders = [];
  bool _isLoading = false;
  String _errorMessage = '';

  List<Order> get orders => _orders;
  bool get isLoading => _isLoading;
  String get errorMessage => _errorMessage;

  final ApiService _apiService = ApiService();

  Future<void> fetchOrders(int userId) async {
    _isLoading = true;
    notifyListeners();

    try {
      _orders = await _apiService.getOrders(userId);
      _errorMessage = '';
    } catch (e) {
      _errorMessage = e.toString();
    }

    _isLoading = false;
    notifyListeners();
  }

  Future<void> addOrder(Order order) async {
    _isLoading = true;
    notifyListeners();

    try {
      final newOrder = await _apiService.addOrder(order);
      _orders.add(newOrder);
      _errorMessage = '';
    } catch (e) {
      _errorMessage = e.toString();
    }

    _isLoading = false;
    notifyListeners();
  }
}

```

4.uas3

a.detail_page.dart(kosong)

b.home_page.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

```

```

import '../models/coffee_model.dart';
import '../providers/coffee_provider.dart';

class CoffeeManagementPage extends StatefulWidget {
  const CoffeeManagementPage({super.key});

  @override
  State<CoffeeManagementPage> createState() => _CoffeeManagementPageState();
}

class _CoffeeManagementPageState extends State<CoffeeManagementPage> {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      Provider.of<CoffeeProvider>(context, listen: false).fetchCoffees();
    });
  }

  @override
  Widget build(BuildContext context) {
    final coffeeProvider = Provider.of<CoffeeProvider>(context);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Manage Coffees'),
        actions: [
          IconButton(
            icon: const Icon(Icons.add),
            onPressed: () {
              _showAddEditCoffeeDialog(context);
            },
          ),
        ],
      ),
      body: coffeeProvider.isLoading
        ? const Center(child: CircularProgressIndicator())
        : coffeeProvider.errorMessage.isNotEmpty
        ? Center(child: Text(coffeeProvider.errorMessage))
        : ListView.builder(
            itemCount: coffeeProvider.coffees.length,
            itemBuilder: (context, index) {
              final coffee = coffeeProvider.coffees[index];
              return ListTile(
                leading: Image.asset(coffee.image, width: 50, height: 50),
                title: Text(coffee.name),
                subtitle: Text('Rp${coffee.price}'),
                trailing: Row(
                  mainAxisAlignment: MainAxisAlignment.min,
                  children: [
                    IconButton(

```

```

        icon: const Icon(Icons.edit),
        onPressed: () {
          _showAddEditCoffeeDialog(context, coffee: coffee);
        },
      ),
      IconButton(
        icon: const Icon(Icons.delete),
        onPressed: () {
          _showDeleteDialog(context, coffee.id);
        },
      ),
    ],
  ),
  onTap: () {
    _showAddEditCoffeeDialog(context, coffee: coffee);
  },
);
},
),
);

void _showAddEditCoffeeDialog(BuildContext context, {Coffee? coffee}) {
  final nameController = TextEditingController(text: coffee?.name ?? '');
  final descriptionController = TextEditingController(text: coffee?.description
  ?? '');
  final priceController = TextEditingController(text: coffee?.price.toString() ??
  '');
  final imageController = TextEditingController(text: coffee?.image ?? '');

  showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: Text(coffee == null ? 'Add Coffee' : 'Edit Coffee'),
        content: SingleChildScrollView(
          child: Column(
            children: [
              TextField(
                controller: nameController,
                decoration: const InputDecoration(labelText: 'Name'),
              ),
              TextField(
                controller: descriptionController,
                decoration: const InputDecoration(labelText: 'Description'),
                maxLines: 3,
              ),
              TextField(
                controller: priceController,
                decoration: const InputDecoration(labelText: 'Price'),
                keyboardType: TextInputType.number,

```

```

        ),
        TextField(
            controller: imageController,
            decoration: const InputDecoration(labelText: 'Image Path'),
        ),
    ],
),
),
actions: [
    TextButton(
        onPressed: () => Navigator.pop(context),
        child: const Text('Cancel'),
    ),
    ElevatedButton(
        onPressed: () async {
            final newCoffee = Coffee(
                id: coffee?.id ?? 0,
                name: nameController.text,
                description: descriptionController.text,
                price: int.tryParse(priceController.text) ?? 0,
                image: imageController.text,
            );

            if (coffee == null) {
                await Provider.of<CoffeeProvider>(context, listen: false)
                    .addCoffee(newCoffee);
            } else {
                await Provider.of<CoffeeProvider>(context, listen: false)
                    .updateCoffee(newCoffee);
            }

            Navigator.pop(context);
        },
        child: Text(coffee == null ? 'Add' : 'Update'),
    ),
],
);
},
);
}

void _showDeleteDialog(BuildContext context, int id) {
    showDialog(
        context: context,
        builder: (context) {
            return AlertDialog(
                title: const Text('Delete Coffee'),
                content: const Text('Are you sure you want to delete this coffee?'),
                actions: [
                    TextButton(
                        onPressed: () => Navigator.pop(context),

```

```

        child: const Text('Cancel'),
      ),
      ElevatedButton(
        onPressed: () async {
          await Provider.of<CoffeeProvider>(context, listen: false)
            .deleteCoffee(id);
          Navigator.pop(context);
        },
        style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
        child: const Text('Delete'),
      ),
    ],
  );
},
);
}
}
}

```

c.login_page.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/auth_provider.dart';

class LoginScreen extends StatelessWidget {
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();

  LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final authProvider = Provider.of<AuthProvider>(context);

    return Scaffold(
      body: Column(
        children: [
          // Gambar atas
          SizedBox(
            height: MediaQuery.of(context).size.height * 0.4,
            child: Image.asset(
              'assets/caffe.jpg',
              fit: BoxFit.cover,
              width: double.infinity,
            ),
          ),
          // Form login
          Expanded(
            child: Container(
              decoration: const BoxDecoration(
                color: Color(0xFF6B4F3B),

```

```
borderRadius: BorderRadius.only(
  topLeft: Radius.circular(30),
  topRight: Radius.circular(30),
),
),
padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 25),
width: double.infinity,
child: SingleChildScrollView(
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text(
        "Welcome",
        style: TextStyle(
          color: Colors.amber,
          fontSize: 22,
          fontWeight: FontWeight.bold,
        ),
      ),
      const Text(
        "Coffee is like a HUG in a MUG",
        style: TextStyle(
          color: Colors.white70,
          fontSize: 14,
        ),
      ),
      const SizedBox(height: 20),
      TextField(
        controller: _emailController,
        decoration: InputDecoration(
          filled: true,
          fillColor: Colors.white70,
          hintText: 'Email',
          border: OutlineInputBorder(
            borderRadius: BorderRadius.all(Radius.circular(20)),
          ),
        ),
      ),
      const SizedBox(height: 15),
      TextField(
        controller: _passwordController,
        obscureText: true,
        decoration: InputDecoration(
          filled: true,
          fillColor: Colors.white70,
          hintText: 'Password',
          border: OutlineInputBorder(
            borderRadius: BorderRadius.all(Radius.circular(20)),
          ),
        ),
      ),
    ],
  ),
),
```

```

const SizedBox(height: 15),
if (authProvider.errorMessage.isNotEmpty)
  Padding(
    padding: const EdgeInsets.only(bottom: 10),
    child: Text(
      authProvider.errorMessage,
      style: const TextStyle(color: Colors.red),
    ),
  ),
Center(
  child: authProvider.isLoading
    ? const CircularProgressIndicator()
    : ElevatedButton(
        onPressed: () async {
          final success = await authProvider.login(
            _emailController.text,
            _passwordController.text,
          );
          if (success) {
            Navigator.pushReplacementNamed(context, '/home');
          }
        },
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.amber,
          foregroundColor: Colors.black,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(20),
          ),
        ),
        child: const Text('Sign in'),
      ),
),
const SizedBox(height: 15),
Center(
  child: TextButton(
    onPressed: () {
      Navigator.pushReplacementNamed(context, '/register');
    },
    child: const Text.rich(
      TextSpan(
        children: [
          TextSpan(
            text: 'New user? ',
            style: TextStyle(color: Colors.white70),
          ),
          TextSpan(
            text: 'Sign up',
            style: TextStyle(color: Colors.amber),
          ),
        ],
      ),
    ),
  ),
),

```

```

    ),
    ),
    ),
  ],
),
),
),
),
),
],
),
);
}
}

```

d.order_form_page.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/auth_provider.dart';
import '../providers/order_provider.dart';
import '../models/order_model.dart';
import 'order_history_page.dart'; // Pastikan path benar

class OrderFormPage extends StatefulWidget {
  final String itemName;
  final String imagePath;
  final int total;

  const OrderFormPage({
    super.key,
    required this.itemName,
    required this.imagePath,
    required this.total,
  });

  @override
  State<OrderFormPage> createState() => _OrderFormPageState();
}

class _OrderFormPageState extends State<OrderFormPage> {
  final _formKey = GlobalKey<FormState>();
  DateTime? selectedDate;
  String? selectedPayment;
  final nameController = TextEditingController();
  final emailController = TextEditingController();

  void submitOrder() {
    if (_formKey.currentState!.validate() && selectedDate != null &&
selectedPayment != null) {
      final authProvider = Provider.of<AuthProvider>(context, listen: false);
      final orderProvider = Provider.of<OrderProvider>(context, listen: false);

```



```

final order = Order(
  id: 0,
  userId: authProvider.user!.id!,
  itemName: widget.itemName,
  imagePath: widget.imagePath,
  total: widget.total,
  date: selectedDate!,
  paymentMethod: selectedPayment!,
);

orderProvider.addOrder(order).then((_) {
  Navigator.pushReplacement(
    context,
    MaterialPageRoute(
      builder: (context) => OrderHistoryPage(
        name: nameController.text,
        email: emailController.text,
        date: selectedDate!,
        itemName: widget.itemName,
        total: widget.total,
        imagePath: widget.imagePath,
      ),
    ),
  );
});
} else {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text("Mohon lengkapi semua data")),
  );
}
}

@override
void dispose() {
  nameController.dispose();
  emailController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Order Form')),
    body: Form(
      key: _formKey,
      child: Column(
        children: [
          TextFormField(
            controller: nameController,
            decoration: const InputDecoration(labelText: 'Nama'),

```

```

        validator: (value) => value!.isEmpty ? 'Nama wajib diisi' : null,
      ),
      TextFormField(
        controller: emailController,
        decoration: const InputDecoration(labelText: 'Email'),
        validator: (value) => value!.isEmpty ? 'Email wajib diisi' : null,
      ),
      ElevatedButton(
        onPressed: submitOrder,
        child: const Text('Pesan'),
      ),
    ],
  ),
),
);
}
}

```

d.order_history_page.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/order_provider.dart';
import '../providers/auth_provider.dart';

class OrderHistoryPage extends StatefulWidget {
  const OrderHistoryPage({
    Key? key,
    required this.name,
    required this.email,
    required this.date,
    required this.itemName,
    required this.total,
    required this.imagePath,
  }) : super(key: key);

  final String name;
  final String email;
  final DateTime date;
  final String itemName;
  final int total;
  final String imagePath;

  @override
  State<OrderHistoryPage> createState() => _OrderHistoryPageState();
}

class _OrderHistoryPageState extends State<OrderHistoryPage> {
  @override
  void initState() {
    super.initState();
  }
}

```

```

WidgetsBinding.instance.addPostFrameCallback((_) {
  final authProvider = Provider.of<AuthProvider>(context, listen: false);
  Provider.of<OrderProvider>(context, listen:
false).fetchOrders(authProvider.user!.id!);
});
}

@override
Widget build(BuildContext context) {
  final orderProvider = Provider.of<OrderProvider>(context);

  return Scaffold(
    backgroundColor: const Color(0xFF6B4F3B),
    appBar: AppBar(
      backgroundColor: const Color(0xFF6B4F3B),
      title: const Text("Order History", style: TextStyle(color: Colors.yellow)),
      iconTheme: const IconThemeData(color: Colors.black),
    ),
    body: orderProvider.isLoading
      ? const Center(child: CircularProgressIndicator())
      : orderProvider.errorMessage.isNotEmpty
        ? Center(child: Text(orderProvider.errorMessage))
        : ListView.builder(
            itemCount: orderProvider.orders.length,
            itemBuilder: (context, index) {
              final order = orderProvider.orders[index];
              return Padding(
                padding: const EdgeInsets.all(8.0),
                child: Container(
                  padding: const EdgeInsets.all(10),
                  color: const Color(0xFFE6B23A),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Row(
                        children: [
                          Image.asset(
                            order.imagePath,
                            width: 135,
                            height: 198,
                            fit: BoxFit.cover,
                          ),
                          const SizedBox(width: 10),
                          Expanded(
                            child: Column(
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: [
                                Row(
                                  children: [
                                    const Icon(Icons.check_circle, color:
Colors.green),

```

```

        const SizedBox(width: 5),
        Text(
          'Order Complete',
          style: TextStyle(
            fontWeight: FontWeight.bold,
            fontSize: 15,
            color: Colors.brown[800],
          ),
        ),
      ],
    ),
    const SizedBox(height: 10),
    Text(
      order.itemName,
      style: TextStyle(
        fontSize: 15,
        color: Colors.brown[800],
      ),
    ),
    Text(
      'Total Rp${order.total}',
      style: TextStyle(
        fontSize: 15,
        color: Colors.brown[800],
      ),
    ),
  ],
),
],
),
const SizedBox(height: 10),
Text(
  'Tanggal order: ${order.date.day}-
${order.date.month}-${order.date.year}',
  style: const TextStyle(color: Colors.white),
),
Text(
  'Metode pembayaran: ${order.paymentMethod}',
  style: const TextStyle(color: Colors.white),
),
],
),
),
),
),
),
),
);
},
),
);
}
}
}

```

e.register_page.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/auth_provider.dart';

class RegisterScreen extends StatelessWidget {
  final TextEditingController _nameController = TextEditingController();
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();

  RegisterScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final authProvider = Provider.of<AuthProvider>(context);

    return Scaffold(
      body: Column(
        children: [
          // Gambar atas
          SizedBox(
            height: MediaQuery.of(context).size.height * 0.3,
            child: Image.asset(
              'assets/caffe.jpg',
              fit: BoxFit.cover,
              width: double.infinity,
            ),
          ),
          // Form register
          Expanded(
            child: Container(
              decoration: const BoxDecoration(
                color: Color(0xFF6B4F3B),
                borderRadius: BorderRadius.only(
                  topLeft: Radius.circular(30),
                  topRight: Radius.circular(30),
                ),
              ),
              padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 25),
              width: double.infinity,
              child: SingleChildScrollView(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    const Text(
                      "Create Account",
                      style: TextStyle(
                        color: Colors.amber,
                        fontSize: 22,
                        fontWeight: FontWeight.bold,
                      ),
                    ),

```

```

    ),
    const Text(
      "Join us for the best coffee experience",
      style: TextStyle(
        color: Colors.white70,
        fontSize: 14,
      ),
    ),
  ),
  const SizedBox(height: 20),
  TextField(
    controller: _nameController,
    decoration: const InputDecoration(
      filled: true,
      fillColor: Colors.white70,
      hintText: 'Full Name',
      border: OutlineInputBorder(
        borderRadius: BorderRadius.all(Radius.circular(20)),
      ),
    ),
  ),
  const SizedBox(height: 15),
  TextField(
    controller: _emailController,
    decoration: const InputDecoration(
      filled: true,
      fillColor: Colors.white70,
      hintText: 'Email',
      border: OutlineInputBorder(
        borderRadius: BorderRadius.all(Radius.circular(20)),
      ),
    ),
  ),
  const SizedBox(height: 15),
  TextField(
    controller: _passwordController,
    obscureText: true,
    decoration: const InputDecoration(
      filled: true,
      fillColor: Colors.white70,
      hintText: 'Password',
      border: OutlineInputBorder(
        borderRadius: BorderRadius.all(Radius.circular(20)),
      ),
    ),
  ),
  const SizedBox(height: 15),
  if (authProvider.errorMessage.isNotEmpty)
    Padding(
      padding: const EdgeInsets.only(bottom: 10),
      child: Text(
        authProvider.errorMessage,

```

```

        style: const TextStyle(color: Colors.red),
      ),
    ),
    Center(
      child: authProvider.isLoading
        ? const CircularProgressIndicator()
        : ElevatedButton(
            onPressed: () async {
              final success = await authProvider.register(
                _nameController.text,
                _emailController.text,
                _passwordController.text,
              );
              if (success) {
                Navigator.pushReplacementNamed(context, '/home');
              }
            },
            style: ElevatedButton.styleFrom(
              backgroundColor: Colors.amber,
              foregroundColor: Colors.black,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(20),
              ),
            ),
            child: const Text('Register'),
          ),
    ),
  ),
  const SizedBox(height: 15),
  Center(
    child: TextButton(
      onPressed: () {
        Navigator.pushReplacementNamed(context, '/login');
      },
      child: const Text.rich(
        TextSpan(
          children: [
            TextSpan(
              text: 'Already have an account? ',
              style: TextStyle(color: Colors.white70),
            ),
            TextSpan(
              text: 'Login',
              style: TextStyle(color: Colors.amber),
            ),
          ],
        ),
      ),
    ),
  ),
],
),
),
),
),
),
],
),

```

```

        ),
      ),
    ],
  ),
);
}
}

```

f.splash_page.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/auth_provider.dart';
import 'home_page.dart';
import 'login_page.dart';

class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    _checkLoginStatus();
  }

  Future<void> _checkLoginStatus() async {
    final authProvider = Provider.of<AuthProvider>(context, listen: false);
    await authProvider.loadUser();

    Future.delayed(const Duration(seconds: 2), () {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(
          builder: (_) => authProvider.user != null
            ? CoffeeManagementPage()
            : LoginScreen(),
        ),
      );
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFF6B4F3B),

```



```

body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      const Image(
        image: AssetImage('assets/caffe1.png'),
        height: 180,
      ),
      const SizedBox(height: 20),
      const Text(
        'Coffee Day',
        style: TextStyle(
          color: Colors.amber,
          fontSize: 24,
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 20),
      const CircularProgressIndicator(color: Colors.amber),
    ],
  ),
),
);
}
}

```

g.main.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'providers/auth_provider.dart';
import 'uas3/splash_screen.dart';
import 'uas3/login_page.dart';
import 'uas3/register_page.dart';
import 'package:uas/uas3/home_page.dart';
// import 'package:uas/uas3/coffee_management_page.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        // Tambahkan provider lain di sini jika diperlukan
      ],
      child: const CoffeeApp(),
    ),
  );
}

class CoffeeApp extends StatelessWidget {
  const CoffeeApp({super.key});
}

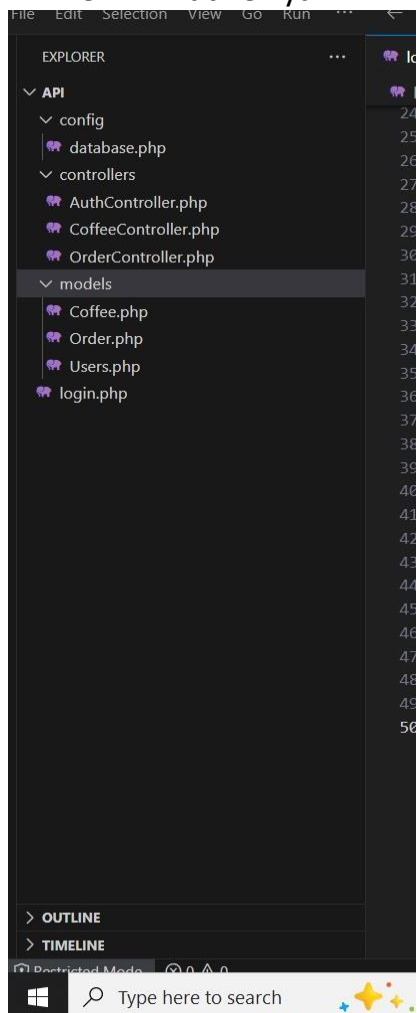
```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Coffe Day',
    theme: ThemeData(
      fontFamily: 'Arial',
      useMaterial3: true,
    ),
    home: const SplashScreen(),
    debugShowCheckedModeBanner: false,
    routes: {
      '/login': (context) => LoginScreen(),
      '/register': (context) => RegisterScreen(),
      // '/home': (context) => const HomePage(),
      '/coffee_management': (context) => const CoffeeManagementPage(),
    },
  );
}
}

```

Ini file PHP native nya



Ini file code nya

1.config

a.database.php

```
<?php
class Database {
    private $host = "localhost";
    private $db_name = "coffee_day";
    private $username = "root";
    private $password = "";
    public $conn;

    public function getConnection() {
        $this->conn = null;
        try {
            $this->conn = new PDO("mysql:host=" . $this->host . ";dbname=" . $this->db_name, $this->username, $this->password);
            $this->conn->exec("set names utf8");
        } catch(PDOException $exception) {
            echo "Connection error: " . $exception->getMessage();
        }
        return $this->conn;
    }
}
```

2.controllers

a.authcontroller.php

```
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: POST");
header("Access-Control-Max-Age: 3600");
header("Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With");

include_once '../config/database.php';
include_once '../models/User.php';

$databse = new Database();
$db = $databse->getConnection();

$user = new User($db);

$data = json_decode(file_get_contents("php://input"));

if($_SERVER['REQUEST_METHOD'] == 'POST') {
    if(isset($data->action)) {
        switch($data->action) {
```

```

        case 'register':
            $user->name = $data->name;
            $user->email = $data->email;
            $user->password = $data->password;

            if($user->emailExists()) {
                http_response_code(400);
                echo json_encode(array("message" => "Email already exists."));
            } else {
                if($user->register()) {
                    http_response_code(201);
                    echo json_encode(array("message" => "User was
registered."));
                } else {
                    http_response_code(503);
                    echo json_encode(array("message" => "Unable to register
user."));
                }
            }
            break;

        case 'login':
            $user->email = $data->email;
            $user->password = $data->password;

            if($user->login()) {
                http_response_code(200);
                echo json_encode(array(
                    "message" => "Login successful",
                    "user_id" => $user->id,
                    "name" => $user->name,
                    "email" => $user->email
                ));
            } else {
                http_response_code(401);
                echo json_encode(array("message" => "Login failed."));
            }
            break;
    }
}
?>

```

b.coffecontroller.php

```

<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE");
header("Access-Control-Max-Age: 3600");

```

```
header("Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers,  
Authorization, X-Requested-With");  
  
include_once '../config/database.php';  
include_once '../models/Coffee.php';  
  
$database = new Database();  
$db = $database->getConnection();  
  
$coffee = new Coffee($db);  
  
$data = json_decode(file_get_contents("php://input"));  
  
if($_SERVER['REQUEST_METHOD'] == 'GET' && isset($_GET['action']) && $_GET['action']  
== 'get_coffees') {  
    $stmt = $coffee->read();  
    $num = $stmt->rowCount();  
  
    if($num > 0) {  
        $coffees_arr = array();  
        while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {  
            extract($row);  
            $coffee_item = array(  
                "id" => $id,  
                "name" => $name,  
                "description" => $description,  
                "price" => $price,  
                "image" => $image  
            );  
            array_push($coffees_arr, $coffee_item);  
        }  
        http_response_code(200);  
        echo json_encode($coffees_arr);  
    } else {  
        http_response_code(404);  
        echo json_encode(array("message" => "No coffees found."));  
    }  
} elseif($_SERVER['REQUEST_METHOD'] == 'POST' && isset($data->action)) {  
    switch($data->action) {  
        case 'add_coffee':  
            $coffee->name = $data->name;  
            $coffee->description = $data->description;  
            $coffee->price = $data->price;  
            $coffee->image = $data->image;  
  
            if($coffee->create()) {  
                http_response_code(201);  
                echo json_encode(array(  
                    "id" => $coffee->id,  
                    "name" => $coffee->name,  
                    "description" => $coffee->description,
```

```

        "price" => $coffee->price,
        "image" => $coffee->image
    ));
} else {
    http_response_code(503);
    echo json_encode(array("message" => "Unable to create coffee."));
}
break;

case 'update_coffee':
    $coffee->id = $data->id;
    $coffee->name = $data->name;
    $coffee->description = $data->description;
    $coffee->price = $data->price;
    $coffee->image = $data->image;

    if($coffee->update()) {
        http_response_code(200);
        echo json_encode(array(
            "id" => $coffee->id,
            "name" => $coffee->name,
            "description" => $coffee->description,
            "price" => $coffee->price,
            "image" => $coffee->image
        ));
    } else {
        http_response_code(503);
        echo json_encode(array("message" => "Unable to update coffee."));
    }
    break;

case 'delete_coffee':
    $coffee->id = $data->id;
    if($coffee->delete()) {
        http_response_code(200);
        echo json_encode(array("message" => "Coffee was deleted."));
    } else {
        http_response_code(503);
        echo json_encode(array("message" => "Unable to delete coffee."));
    }
    break;
}
}
?>

```

c.ordercontroller.php

```

<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: GET, POST");

```

```

header("Access-Control-Max-Age: 3600");
header("Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With");

include_once '../config/database.php';
include_once '../models/Order.php';

$database = new Database();
$db = $database->getConnection();

$order = new Order($db);

if($_SERVER['REQUEST_METHOD'] == 'GET' && isset($_GET['action']) && $_GET['action'] == 'get_orders' && isset($_GET['user_id'])) {
    $stmt = $order->readByUser($_GET['user_id']);
    $num = $stmt->rowCount();

    if($num > 0) {
        $orders_arr = array();
        while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
            extract($row);
            $order_item = array(
                "id" => $id,
                "user_id" => $user_id,
                "item_name" => $item_name,
                "image_path" => $image_path,
                "total" => $total,
                "date" => $date,
                "payment_method" => $payment_method
            );
            array_push($orders_arr, $order_item);
        }
        http_response_code(200);
        echo json_encode($orders_arr);
    } else {
        http_response_code(404);
        echo json_encode(array("message" => "No orders found."));
    }
} elseif($_SERVER['REQUEST_METHOD'] == 'POST' && isset($data->action) && $data->action == 'add_order') {
    $order->user_id = $data->user_id;
    $order->item_name = $data->item_name;
    $order->image_path = $data->image_path;
    $order->total = $data->total;
    $order->date = $data->date;
    $order->payment_method = $data->payment_method;

    if($order->create()) {
        http_response_code(201);
        echo json_encode(array(
            "id" => $order->id,

```

```

        "user_id" => $order->user_id,
        "item_name" => $order->item_name,
        "image_path" => $order->image_path,
        "total" => $order->total,
        "date" => $order->date,
        "payment_method" => $order->payment_method
    ));
} else {
    http_response_code(503);
    echo json_encode(array("message" => "Unable to create order."));
}
}
?>

```

3.models

a.coffee.php

```

<?php
class Coffee {
    private $conn;
    private $table_name = "coffees";

    public $id;
    public $name;
    public $description;
    public $price;
    public $image;

    public function __construct($db) {
        $this->conn = $db;
    }

    public function read() {
        $query = "SELECT * FROM " . $this->table_name;
        $stmt = $this->conn->prepare($query);
        $stmt->execute();
        return $stmt;
    }

    public function create() {
        $query = "INSERT INTO " . $this->table_name . "
            SET name=:name, description=:description, price=:price,
image=:image";

        $stmt = $this->conn->prepare($query);

        $this->name = htmlspecialchars(strip_tags($this->name));
        $this->description = htmlspecialchars(strip_tags($this->description));
        $this->price = htmlspecialchars(strip_tags($this->price));
        $this->image = htmlspecialchars(strip_tags($this->image));
    }
}

```



```

        $stmt->bindParam(":name", $this->name);
        $stmt->bindParam(":description", $this->description);
        $stmt->bindParam(":price", $this->price);
        $stmt->bindParam(":image", $this->image);

        if($stmt->execute()) {
            $this->id = $this->conn->lastInsertId();
            return true;
        }
        return false;
    }

    public function update() {
        $query = "UPDATE " . $this->table_name . "
                SET name=:name, description=:description, price=:price,
image=:image
                WHERE id=:id";

        $stmt = $this->conn->prepare($query);

        $this->name = htmlspecialchars(strip_tags($this->name));
        $this->description = htmlspecialchars(strip_tags($this->description));
        $this->price = htmlspecialchars(strip_tags($this->price));
        $this->image = htmlspecialchars(strip_tags($this->image));
        $this->id = htmlspecialchars(strip_tags($this->id));

        $stmt->bindParam(":name", $this->name);
        $stmt->bindParam(":description", $this->description);
        $stmt->bindParam(":price", $this->price);
        $stmt->bindParam(":image", $this->image);
        $stmt->bindParam(":id", $this->id);

        return $stmt->execute();
    }

    public function delete() {
        $query = "DELETE FROM " . $this->table_name . " WHERE id = ?";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(1, $this->id);
        return $stmt->execute();
    }
}
?>

```

c.order.php

```

<?php
class Order {
    private $conn;
    private $table_name = "orders";

```

```

public $id;
public $user_id;
public $item_name;
public $image_path;
public $total;
public $date;
public $payment_method;

public function __construct($db) {
    $this->conn = $db;
}

public function readByUser($user_id) {
    $query = "SELECT * FROM " . $this->table_name . " WHERE user_id = ?";
    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(1, $user_id);
    $stmt->execute();
    return $stmt;
}

public function create() {
    $query = "INSERT INTO " . $this->table_name . "
        SET user_id=:user_id, item_name=:item_name,
image_path=:image_path,
        total=:total, date=:date, payment_method=:payment_method";

    $stmt = $this->conn->prepare($query);

    $this->user_id = htmlspecialchars(strip_tags($this->user_id));
    $this->item_name = htmlspecialchars(strip_tags($this->item_name));
    $this->image_path = htmlspecialchars(strip_tags($this->image_path));
    $this->total = htmlspecialchars(strip_tags($this->total));
    $this->date = htmlspecialchars(strip_tags($this->date));
    $this->payment_method = htmlspecialchars(strip_tags($this->
>payment_method));

    $stmt->bindParam(":user_id", $this->user_id);
    $stmt->bindParam(":item_name", $this->item_name);
    $stmt->bindParam(":image_path", $this->image_path);
    $stmt->bindParam(":total", $this->total);
    $stmt->bindParam(":date", $this->date);
    $stmt->bindParam(":payment_method", $this->payment_method);

    if($stmt->execute()) {
        $this->id = $this->conn->lastInsertId();
        return true;
    }
    return false;
}
}
?>

```

D.users.php

```
<?php
class User {
    private $conn;
    private $table_name = "users";

    public $id;
    public $name;
    public $email;
    public $password;

    public function __construct($db) {
        $this->conn = $db;
    }

    public function register() {
        $query = "INSERT INTO " . $this->table_name . " SET name=:name,
email=:email, password=:password";
        $stmt = $this->conn->prepare($query);

        $this->name = htmlspecialchars(strip_tags($this->name));
        $this->email = htmlspecialchars(strip_tags($this->email));
        $this->password = password_hash($this->password, PASSWORD_BCRYPT);

        $stmt->bindParam(":name", $this->name);
        $stmt->bindParam(":email", $this->email);
        $stmt->bindParam(":password", $this->password);

        if($stmt->execute()) {
            return true;
        }
        return false;
    }

    public function login() {
        $query = "SELECT id, name, email, password FROM " . $this->table_name . "
WHERE email = ? LIMIT 0,1";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(1, $this->email);
        $stmt->execute();
        $num = $stmt->rowCount();

        if($num > 0) {
            $row = $stmt->fetch(PDO::FETCH_ASSOC);
            if(password_verify($this->password, $row['password'])) {
                $this->id = $row['id'];
                $this->name = $row['name'];
                $this->email = $row['email'];
                return true;
            }
        }
    }
}
```

```

        return false;
    }

    public function emailExists() {
        $query = "SELECT id FROM " . $this->table_name . " WHERE email = ? LIMIT
0,1";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(1, $this->email);
        $stmt->execute();
        $num = $stmt->rowCount();
        return $num > 0;
    }
}
?>

```

Ini bagian login.page(diluar dr folder models)

```

<?php
header("Content-Type: application/json");
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");

require_once 'config/database.php';

// Tangani preflight request
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(200);
    exit;
}

$data = json_decode(file_get_contents("php://input"), true);
$email = $data['email'] ?? '';
$password = $data['password'] ?? '';

if (empty($email) || empty($password)) {
    http_response_code(400);
    echo json_encode(['status' => 'error', 'message' => 'Email and password are
required']);
    exit;
}

try {
    $stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email");
    $stmt->execute(['email' => $email]);
    $user = $stmt->fetch();

    if ($user && password_verify($password, $user['password'])) {
        $response = [
            'status' => 'success',

```

```

        'message' => 'Login successful',
        'user' => [
            'id' => $user['id'],
            'name' => $user['name'],
            'email' => $user['email']
        ]
    ];
    http_response_code(200);
} else {
    $response = ['status' => 'error', 'message' => 'Invalid email or
password'];
    http_response_code(401);
}
echo json_encode($response);
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(['status' => 'error', 'message' => 'Database error: '. $e-
>getMessage()]);
}
?>

```

Bagian xampp

